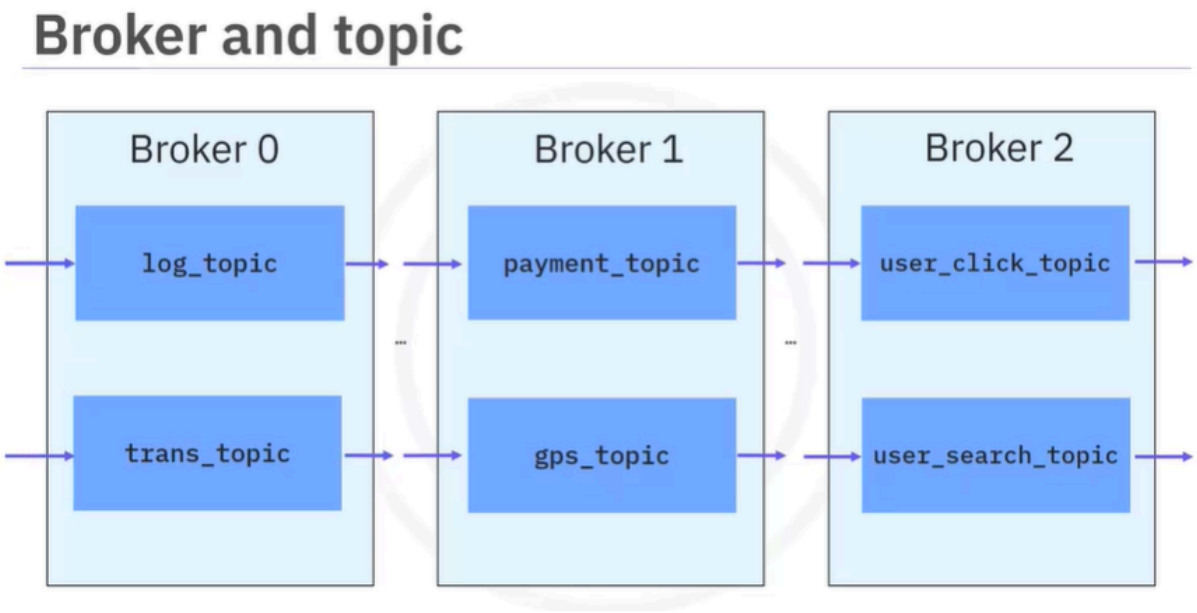
Building Event Streaming Pipelines using Kafka

Sure! Let's talk about the core components of Kafka in simple terms.

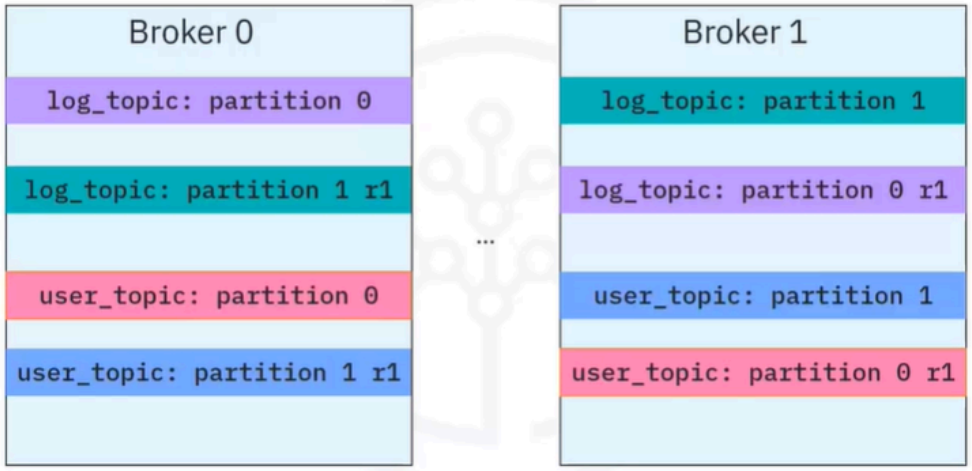
Kafka is like a busy post office that helps send and receive messages (or events) between different applications. At the heart of Kafka are brokers, which are the servers that handle all the messages. You can think of a broker as a dedicated worker at the post office who receives, stores, and distributes letters. These messages are organized into topics, which are like different mailboxes for specific types of messages, such as logs or user activities. Each topic can be divided into smaller sections called partitions, which helps manage the flow of messages and ensures that everything runs smoothly, even if some workers are unavailable.

Now, imagine you have two friends who want to send messages to each other. One friend is a producer, who writes messages and puts them in the right mailbox (topic), while the other friend is a consumer, who reads those messages from the mailbox. The beauty of Kafka is that these two friends can work independently; the producer can send messages whenever they want, and the consumer can read them at their own pace. This way, they can communicate effectively without having to wait for each other.

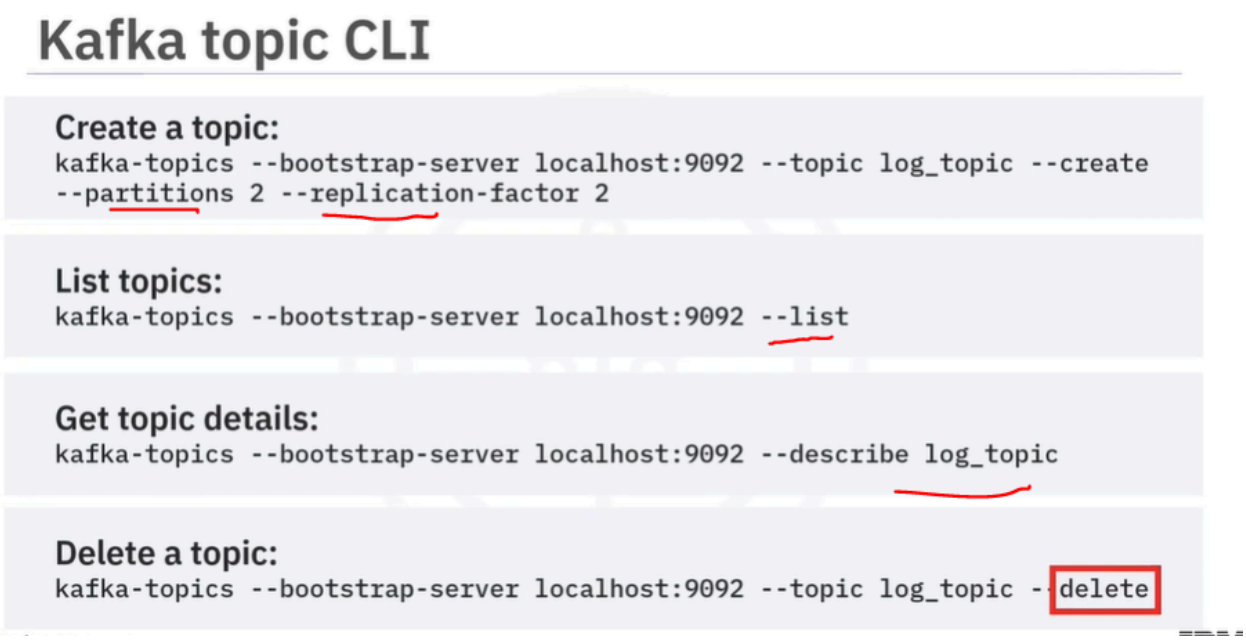


- A Kafka cluster contains one or many brokers. You may think of a Kafka broker as a dedicated server to receive, store, process, and distribute events. Brokers are synchronized and use KRaft controller nodes that use the consensus protocol to manage the Kafka metadata log that contains information about each change to the cluster metadata.
- Let's look at an example of how the data is organized as topics in the brokers. A log_topic and a transaction topic in broker 0, a payment_topic and a gps_topic in broker 1, and a user_click_topic and user_search_topic in broker 2. Each broker contains one or many topics. You can think of a topic as a database to store specific types of events such as logs, transactions, and metrics. Brokers manage to save published events into topics and distribute the events to subscribed consumers.

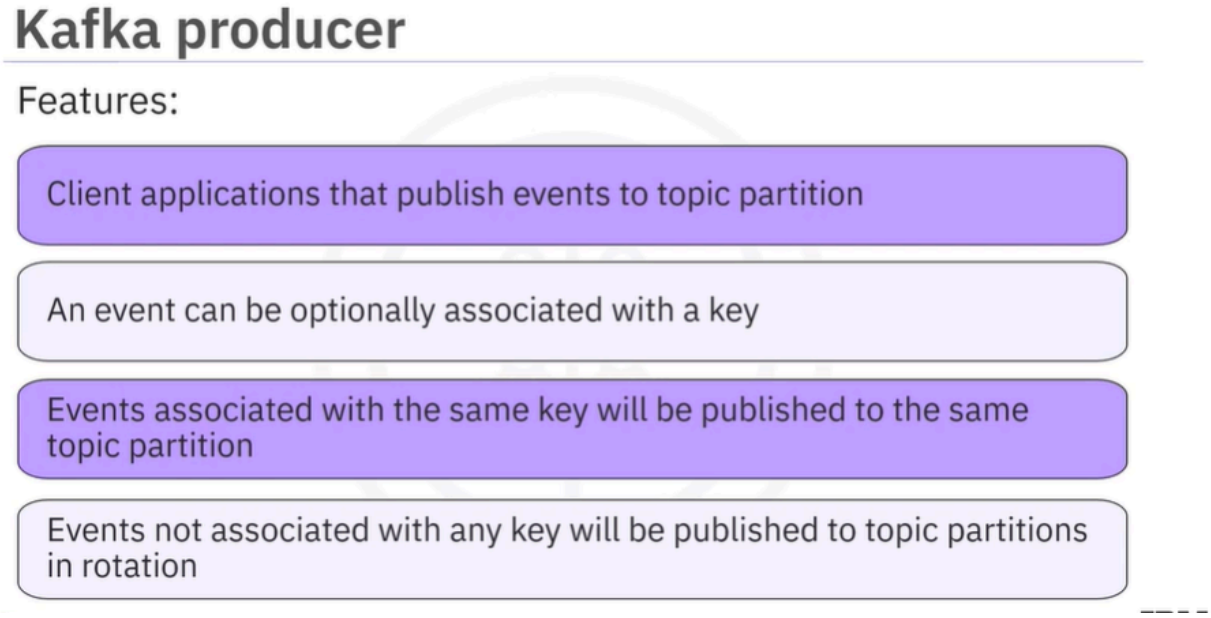
Partition and replications



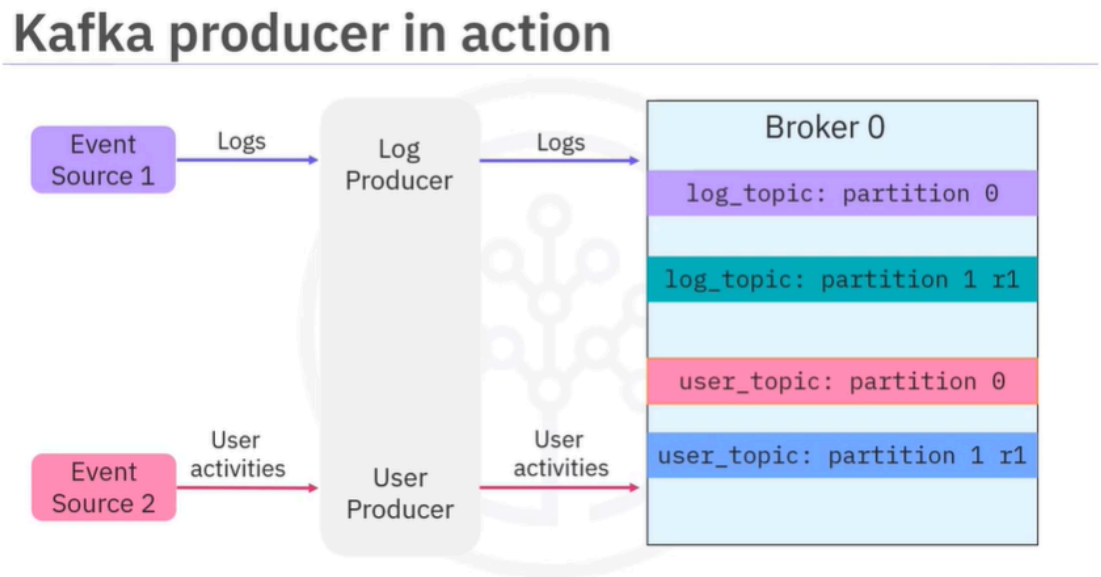
- Like many other distribution systems, Kafka implements the concepts of partitioning and replicating. It uses topic partitions and replications to increase fault tolerance and throughput so that event publication and consumption can be done in parallel with multiple brokers. In addition, even if some brokers are down, kafka clients are still able to work with the target topics replicated in other working brokers.
- For example, a log_topic has been separated into two partitions (0,1) and a user topic has been separated into two partitions (0,1) and each topic partitioned is duplicated into two replications and stored in different brokers.



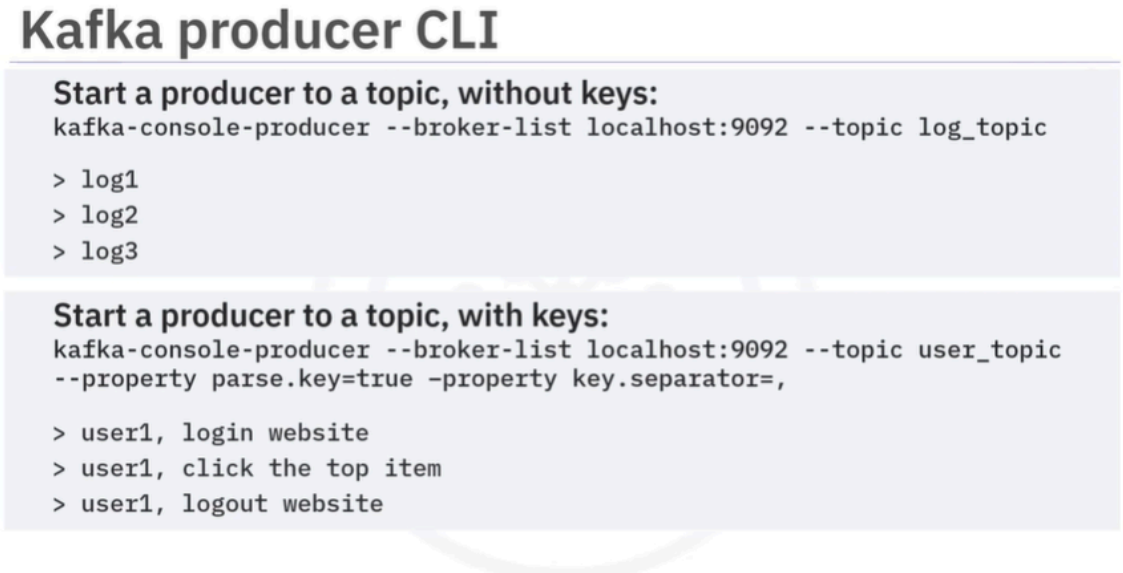
- The Kafka CLI, or command-line interface client, provides a collection of powerful script files for users to build an event streaming pipeline. The Kafka topics script is the one you will be using often to manage topics in a Kafka cluster.
- It is straightforward. Let's have a look at some common usages. The first one is to create a topic. Here we are trying to create a topic called log_topic with two partitions and two replications.
- One important note here is that many Kafka commands like kafka-topics require users to refer to a running Kafka cluster with a host and a port, such as a local host with port 9092. After you have created some topics, you can check all created topics in the cluster using the list option. And if you want to check more details of a topic such as partitions and replications. You can use the describe option and you can delete a topic using the delete option.



- Next you will find out more about publishing events using Kafka producers. Features of a Kafka producer: they are client applications that publish events to topic partitions according to the same order as they are published. When publishing an event in a Kafka producer, an event can be optionally associated with a key. Events associated with the same key will be published to the same topic partition.

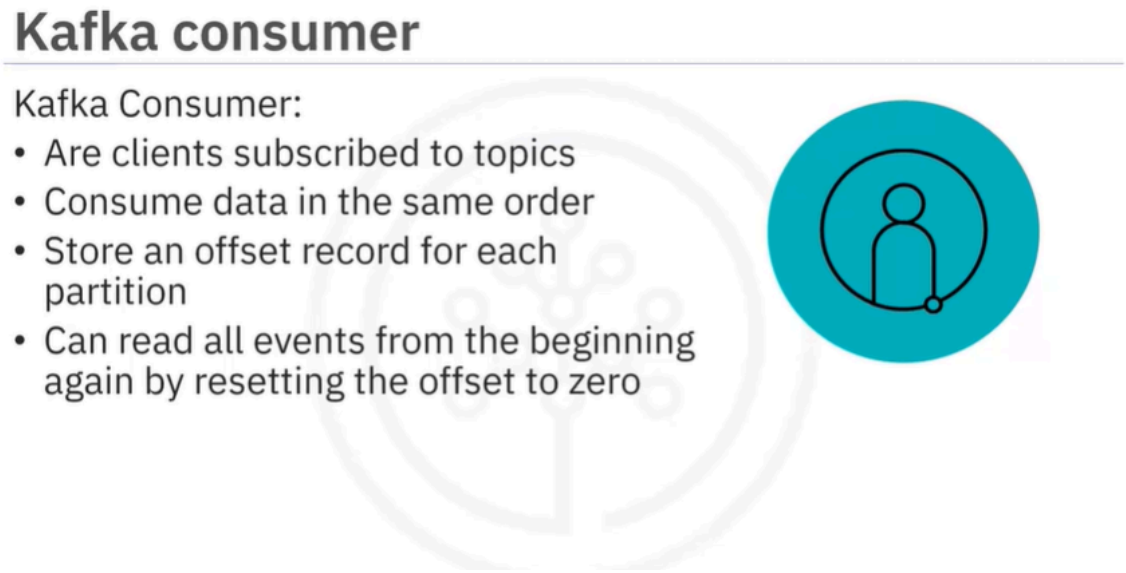


- Events not associated with any key will be published to topic partitions in rotation. Let's see how you can publish events to topic partitions using the following example. Suppose you have an Event Source 1 which generates various log entries and an Event Source 2 which generates user activity tracking records. Then you can create a Kafka producer to publish log records to log topic partitions and a user producer to publish user activity events to user topic partitions, respectively. When you publish events in producers, you can choose to associate events with a key, for example, an application name or a user ID.



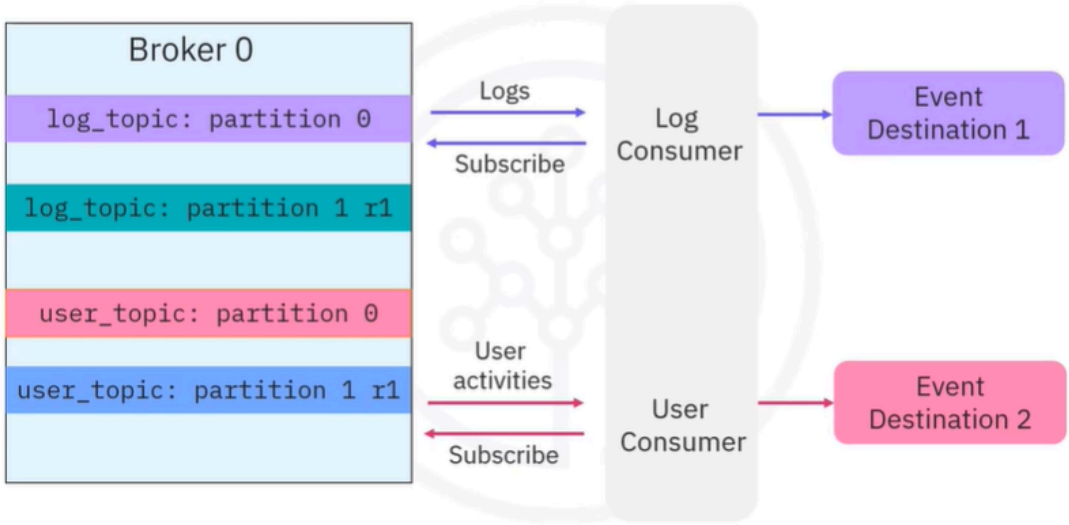
- Similar to the Kafka topic CLI, Kafka provides the Kafka producer CLI for users to manage producers. The most important aspect is starting a producer to write or publish events to a topic. Here you start a producer and point it to the log_topic. Then you can type some messages in the console to start publishing events. For example, log1, log2, and log3.
- You can provide keys to events to make sure the events with the same key will go to the same partition. Here you are starting a producer to user_topic with the parse.key option to be true and you also specify the key.separator to be comma.

- Then you can write messages as follows, key, user1, value login website, key, user1, value, click the top item and key, user1, value, logout website. Accordingly, all events about user one will be saved in the same partition to facilitate the reading for consumers.



- Once events are published and properly stored in topic partitions, you can create consumers to read them. Consumers are client applications that can subscribe to topics and read the stored events. Then event destinations can further read events from Kafka consumers. Consumers read data from topic partitions in the same order as they are published. Consumers also store an offset for each topic partition as the last read position. With the offset, consumers are guaranteed to read events as they occur.
- A playback is also possible by resetting the offset to zero. This way, the consumer can read all events in the topic partition from the beginning. In Kafka, producers and consumers are fully decoupled. As such, producers don't need to synchronize with consumers, and after events are stored in topics, consumers can have independent schedules to consume them.

Kafka consumer in action



- To read published log and user events from topic partitions, you will need to create log and user consumers and make them subscribe to corresponding topics. Then Kafka will push the events to those subscribed consumers. Then the consumers will further send to event destinations.

Kafka consumer CLI

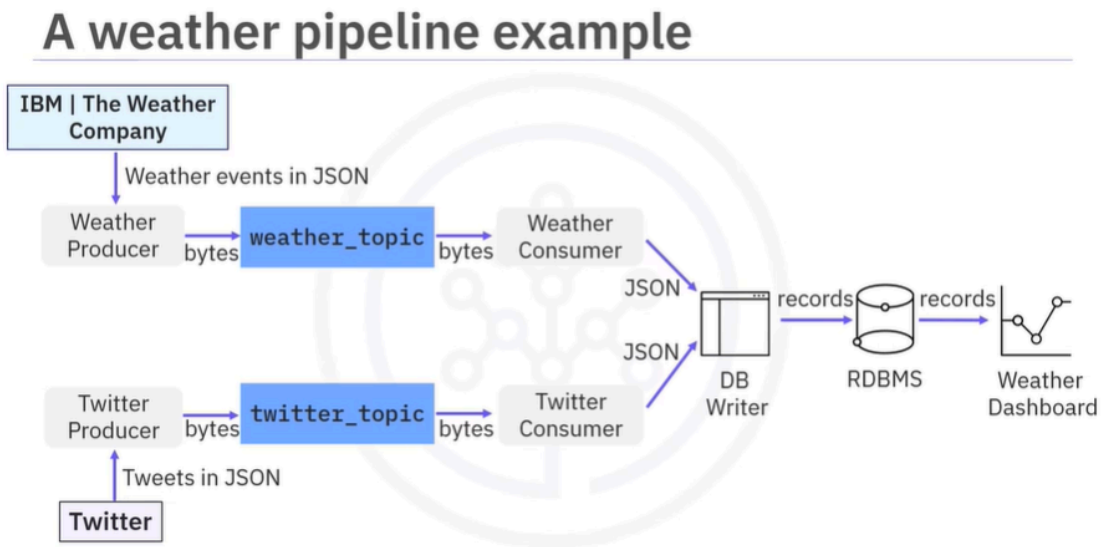
Read new events from the *log_topic, offset 1*:

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic log_topic

> (offset 2) log3
> (offset 3) log4
```

Read all events from the *log_topic, offset 0*:

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic log_topic
--from-beginning

> (offset 0) log1
> (offset 1) log2
> (offset 2) log3
> (offset 3) log4
```

- To start a consumer is also easy using the Kafka consumer script. Let's read events from the log_topic.
- You just need to run the Kafka console consumer script and specify a Kafka cluster and the topic to subscribe to. Here you can subscribe to and read events from the topic log_topic. Then the started consumer will read only the new events starting from the last partition offset. After those events are consumed, the partition offset for the consumer will also be updated and committed back to Kafka. Very often a user wants to read all events from the beginning as a playback of all historical events. To do so, you just need to add the from-beginning option. Now you can read all events starting from offset 0.



- Let's have a look at a more concrete example to help you understand how to build an event streaming pipeline end to end. Suppose you want to collect and analyze weather and Twitter event streams so that you can correlate how people talk about extreme weather on Twitter.
- Here you can use two event sources: IBM weather API to obtain real time and forecasted weather data in JSON format. Twitter API to obtain real-time tweets and mentions also in JSON format.
- To receive weather and Twitter JSON data in Kafka, you then create a weather topic and a Twitter topic in a Kafka cluster with some partitions and

replications.

- To publish weather and Twitter JSON data to the two topics, you need to create a weather producer and a Twitter producer.
- The event's JSON data will be serialized into bytes and saved in Kafka topics. To read events from the two topics, you need to create a weather consumer and a Twitter consumer. The bytes stored in Kafka topics will be deserialized into event JSON data.
- If you now want to transport the weather and Twitter event JSON data from the consumers to a relational database, you will use a DB writer to parse those JSON files and create database records, and then you can write those records into a database using SQL insert statements.
- Finally, you can query the database records from the relational database and visualize and analyze them in a dashboard to complete the end-to-end pipeline.

Recap

In this video, you learned that:

- The core components of Kafka are:
  - Brokers: The dedicated servers to receive, store, process, and distribute events
  - Topics: The containers or databases of events
  - Partitions: Divide topics into different brokers
  - Replications: Duplicate partitions into different brokers
  - Producers: Kafka client applications that publish events into topics
  - Consumers: Kafka client applications that subscribe to topics and read events from them
- The Kafka-topics CLI manages topics
- The Kafka-console-producer CLI manages producers
- The Kafka-console-consumer manages consumers